



# Tecnológico de Monterrey

**Act 2.3 - Actividad Integral estructura de datos lineales**

**Daniel Salazar Domínguez - A01564550**

**Fecha de entrega: 11/10/2026**

## Reflexión

Esta actividad pide un código capaz de leer un archivo de texto y ordenar la información utilizando las IPs. Yo reutilice un código que pedía algo similar pero ordenándolo por fechas, y lo edité para que lo hiciera por IPs usando un ordenamiento Mergesort para ordenarlo de manera ascendente en un archivo de texto nuevo ya ordenado, aunque en el código a la hora de desplegar rango lo despliega de manera descendente. Por la misma razón del código de la actividad anterior 1.3, el nuevo archivo es ascendente por limpieza y el despliegue descendente por requerimientos de la rúbrica de diseño.

No se consideraron algoritmos de ordenamiento aparte del Mergesort, ya que en la **Actividad 1.3** ya se había decidido que el Mergesort mantenía la complejidad temporal en los mejores y peores casos. Este algoritmo tiene como propósito ordenar, y su complejidad temporal es  $O(n \log n)$ . Se puede apreciar mejor en la siguiente tabla:

**Tabla de complejidad computacional:**

### Array Sorting Algorithms

| Algorithm                      | Time Complexity     |                        |                   | Space Complexity |
|--------------------------------|---------------------|------------------------|-------------------|------------------|
|                                | Best                | Average                | Worst             | Worst            |
| <a href="#">Quicksort</a>      | $\Omega(n \log(n))$ | $\Theta(n \log(n))$    | $O(n^2)$          | $O(\log(n))$     |
| <a href="#">Mergesort</a>      | $\Omega(n \log(n))$ | $\Theta(n \log(n))$    | $O(n \log(n))$    | $O(n)$           |
| <a href="#">Timsort</a>        | $\Omega(n)$         | $\Theta(n \log(n))$    | $O(n \log(n))$    | $O(n)$           |
| <a href="#">Heapsort</a>       | $\Omega(n \log(n))$ | $\Theta(n \log(n))$    | $O(n \log(n))$    | $O(1)$           |
| <a href="#">Bubble Sort</a>    | $\Omega(n)$         | $\Theta(n^2)$          | $O(n^2)$          | $O(1)$           |
| <a href="#">Insertion Sort</a> | $\Omega(n)$         | $\Theta(n^2)$          | $O(n^2)$          | $O(1)$           |
| <a href="#">Selection Sort</a> | $\Omega(n^2)$       | $\Theta(n^2)$          | $O(n^2)$          | $O(1)$           |
| <a href="#">Tree Sort</a>      | $\Omega(n \log(n))$ | $\Theta(n \log(n))$    | $O(n^2)$          | $O(n)$           |
| <a href="#">Shell Sort</a>     | $\Omega(n \log(n))$ | $\Theta(n(\log(n))^2)$ | $O(n(\log(n))^2)$ | $O(1)$           |
| <a href="#">Bucket Sort</a>    | $\Omega(n+k)$       | $\Theta(n+k)$          | $O(n^2)$          | $O(n)$           |
| <a href="#">Radix Sort</a>     | $\Omega(nk)$        | $\Theta(nk)$           | $O(nk)$           | $O(n+k)$         |
| <a href="#">Counting Sort</a>  | $\Omega(n+k)$       | $\Theta(n+k)$          | $O(n+k)$          | $O(k)$           |
| <a href="#">Cubesort</a>       | $\Omega(n)$         | $\Theta(n \log(n))$    | $O(n \log(n))$    | $O(n)$           |

- **MergeSort** consiste en dividir la lista en la mitad varias veces hasta convertir las divisiones en elementos individuales, luego se rearmen los grupos, de la misma manera que se dividieron, comparando los valores y ordenarlos de manera correcta hasta ordenar toda la lista.

Los valores de la tabla anterior indican la complejidad temporal (cuanto tardan en ordenar) de cada algoritmo, como ya mencioné previamente se eligió Mergesort porque su complejidad era igual en los mejores y peores casos.

Para este problema se utilizó la búsqueda secuencial de la **Actividad 1.3**, la cual busca 1 por 1 desde el final hasta el inicio del array, que es menos eficiente que binario porque busca de uno por uno y no descarta valores. Igual que en la **Actividad 1.3** utilice búsqueda secuencial, ya que era más simple de implementar en el programa ya que buscaba rangos de valores y los desplegaba de manera descendente.

En general, mi programa permite ordenar un archivo de texto con bitácoras en el siguiente formato: mes en formato de 3 letras, día, hora en formato hh:mm:ss, dirección IP origen en formato ####.###.####.####:##### y la razón de falla. El programa le da peso a las direcciones de IP, sin utilizar el puerto, y les asigna valores long long para que se pueda comparar utilizando potencias de 1000 ( $1000^3$ ,  $1000^2$ ,  $1000^1$  y  $1000^0$ ). Y sirve para crear un nuevo archivo de texto que entrega la bitácora ordenada de manera ascendente.

### **Lista doblemente ligada**

La lista doblemente ligada, consiste en una lista con nodos, en la cual cada nodo apunta al nodo anterior y posterior.

#### **Ventajas**

- Permite recorrer la lista de manera bidireccional.
- Es una estructura de datos fácil de implementar.
- Borrar nodos es más sencillo que en una lista enlazada simple.

#### **Desventajas**

- Usa más memoria que un array o lista enlazada simple.
- Los elementos están almacenados de manera aleatoria en la memoria y no se pueden acceder de manera directa.
- Moverse en la lista doblemente ligada puede ser más lento que en una lista enlazada simple.

Yo no utilice una lista doblemente ligada, sino un vector (arreglo dinámico cuyo tamaño que puede crecer y decrecer después de ser creado). Considero que es más apropiado para el problema, ya que es más flexible al agregar elementos al vector si la bitácora llegara a cambiar.

A continuación se muestra la complejidad de las operaciones del programa utilizando el vector y una lista doblemente ligada:

| Operación solicitada                    | Vector        | Lista doblemente ligada |
|-----------------------------------------|---------------|-------------------------|
| Leer el archivo y guardar los datos     | $O(n)$        | $O(n)$                  |
| Ordenar usando MergeSort                | $O(n \log n)$ | $O(n \log n)$           |
| Desplegar los IPs de manera descendente | $O(n)$        | $O(n)$                  |
| Crear y guardar el archivo ordenado     | $O(n)$        | $O(n)$                  |

Observando la tabla se puede observar que la complejidad temporal es la misma usando ambas estructuras de datos, pero yo no consideraba necesario usar la lista doblemente ligada, ya que no era necesario acceder a elementos anteriores, sino más bien avanzar en la lista y registrar los datos o desplegarlos. En este tipo de problema y en mi programa nunca se necesita acceder de manera bidireccional a la lista para consultar los datos, sino que se analiza la lista de manera secuencial.

Para resumir el uso de la lista doblemente ligada no fue considerada necesaria, debido a el uso adicional de memoria comparándolo con el vector, y no era necesario aprovechar las ventajas que ofrecía la lista doblemente ligada para resolver este problema usando una solución simple con vectores.

## Fuentes bibliográficas

*Big-O Algorithm Complexity Cheat Sheet (Know Thy Complexities!) @ericdrowell.* (s. f.).

Recuperado 7 de octubre de 2026, de <https://www.bigocheatsheet.com/>

GeeksforGeeks. (2025, 23 de julio). *Advantages, disadvantages, and uses of doubly linked*

*list.* Recuperado 7 de octubre de 2026, de

<https://www.geeksforgeeks.org/dsa/advantages-disadvantages-and-uses-of-doubly-linked-list/>

GeeksforGeeks. (2026, 20 enero). *Sorting algorithms.* GeeksforGeeks. Recuperado 7 de

octubre de 2026, de <https://www.geeksforgeeks.org/dsa/sorting-algorithms/>

This vs. That. (s. f.). *Doubly linked list vs. singly linked list—What's the difference?*

Recuperado el 7 de octubre de 2026, de

<https://thisvsthat.io/doubly-linked-list-vs-singly-linked-list>